MaxxLoop

Complete Project Documentation

Focus & Recovery / Personal Capacity Learning Loop

This document is generated from the current repository documentation.
It covers product behavior, architecture, data, authentication, APIs,
local development, deployment gaps, hackathon presentation, and limits.
It describes MaxxLoop as a wellness/productivity prototype, not a medical device.

Contents

01. Project Overview
02. Product
03. User Flow
04. Architecture
05. Technical Architecture
06. Frontend
07. Backend
08. API Reference
09. Database
10. Data Model
11. Authentication
12. The MaxxLoop Loop
13. Recommender
14. Insights
15. Security
16. Privacy
17. Testing
18. Development
19. Environment
20. Deployment
21. Troubleshooting
22. Hackathon Guide
23. Demo Script
24. Judge Q&A
25. Limitations
26. Roadmap

Project Overview

In one sentence

MaxxLoop turns a short focus/energy/stress check-in into an explanation, one small suggested action, and a follow-up record that can inform future suggestions.

30-second explanation

Students and knowledge workers can notice that they feel overloaded without knowing what small step to take next or whether it helped. MaxxLoop compares a user's check-in with available recent signals, explains the main contributors, offers one action from a curated library, and asks for another check-in. Feedback is saved so later recommendations can adapt. It is a wellness/productivity prototype, not a medical device or clinical diagnosis system.

One-minute explanation

MaxxLoop is designed around a personal learning loop: Track, Understand, Act, Measure, Improve. A person reports focus, energy, and stress on a short check-in. The API combines those values with available activity signals and historical baselines to compute a bounded capacity estimate. It identifies contributors, applies drop/cooldown rules, and may recommend one action. After the action window, the user can report their state again and whether the action felt helpful. The system records the before/after difference, an estimated no-action drift, and feedback. A statistical recommender uses prior outcomes to adjust which actions it samples. These are self-report measurements and estimates; they do not establish medical status or causal effectiveness.

Technical explanation

The web client is Next.js App Router with React and TypeScript. It calls a FastAPI API using credentialed HTTP requests and a session cookie. MongoDB stores account/session records; SQLModel persists product records in the configured relational database (SQLite by default). The API's baseline, capacity, driver, drop-detection, recommender, and measurement modules implement deterministic/statistical logic. Explanation text defaults to a template provider, with optional Gemini, Ollama, and OpenAI-compatible providers. See [Architecture], [Recommender], and [API].

Problem and core idea

The product addresses a product-design gap: self-tracking can produce information without an immediate next step, generic advice may not fit the moment, and users may not record whether an action helped them. MaxxLoop connects these stages in one repeatable flow. This is the project's intended value proposition, not a market-wide claim that competitors lack these features.

Audience and output

Initial intended users are students and knowledge workers who want lightweight focus/recovery support. Each loop can produce a capacity estimate, signal-driver labels, an offered action, a measured outcome, and a helpfulness response. Insights summarize records already present for the signed-in user.

What the system learns

For each user/action pair, an ActionPrior stores Beta parameters (alpha, beta). Measurement updates one parameter based on whether the computed net effect crosses a threshold. This is a lightweight online statistical heuristic, not a trained model and not proof that the action caused a change.

Non-medical boundary

MaxxLoop does not diagnose, treat, prescribe, or replace professional care. Its capacity score is a product estimate, not a validated clinical measure. Crisis phrase scanning is limited. See [Privacy] and [Limitations].

Product

Product promise

Help a user move from "I feel off" to one reasonable next action and a follow-up check-in, with a personal history that can guide future suggestions.

Product principles visible in the implementation

- Short self-report check-in: Focus, Energy, Stress (1-5) and optional note.
- Explainability: capacity response includes score, baseline, delta, status, drivers, sparkline, and timestamp.
- One offered micro-action per triggered intervention.
- A measured follow-up and optional helpfulness feedback.
- User history and insights derived from saved records.
- Non-medical language and a limited safety interruption for detected crisis phrases.

Current scope

Implemented routes include home, login, signup, dashboard, demo, insights, history, profile, onboarding, and privacy. The API contains check-in, signals, capacity, loop, insights, privacy, demo, authentication, and health routes. See [Frontend] and [API].

Product terminology

- Signal: one timestamped numeric observation, such as a self-report or imported/manual value.
- Snapshot: computed capacity score and baseline at one moment.
- Intervention: one offered action linked to a snapshot.
- Outcome: before/after scores, estimated drift, net effect, confidence label, and optional feedback.
- Loop: the product journey across check-in, action, follow-up, and learning; it is represented through related records rather than a dedicated Loop database table.

Product non-goals

The current product is not a clinician service, diagnostic tool, emergency monitor, validated clinical intervention, or automatically connected calendar/wearable/browser agent. Any such integrations or clinical use require future implementation, consent, review, and validation.

User Flow

Standard authenticated journey

```
flowchart TD
  A[Home /] --> B{Account session?}
  B -- No --> C[Sign up or log in]
  B -- Yes --> D[Dashboard]
  C --> D
  D --> E[Submit focus, energy, stress check-in]
  E --> F{Crisis phrase detected?}
  F -- Yes --> G[Pause normal flow and show support card]
  F -- No --> H[Save signals and compute snapshot]
  H --> I{Drop and cooldown allows?}
  I -- No --> J[Show capacity/status]
  I -- Yes --> K[Explain and offer one action]
  K --> L[Start or skip]
  L --> M[Follow-up check-in]
  M --> N[Save outcome and update prior]
  N --> O[Helpful/not helpful feedback]
  O --> P[History and insights]
  P --> D
```

Account flow

Signup collects name, email, and password (minimum 8 characters, maximum 128); email is normalized to lowercase. Successful signup creates an account in MongoDB and returns a session cookie. Login uses the same email/password; logout invalidates the session and clears the cookie. The frontend AuthProvider restores current-user state through /auth/me. API access to product data requires an authenticated session.

Check-in and action flow

The dashboard submits Focus/Energy/Stress values to POST /checkins. The backend scans optional note text for configured crisis phrases before storing the check-in. If not intercepted, it saves three numeric signals, gets latest additional signals, computes capacity and drivers, checks drop/cooldown, and may create an intervention. The UI displays the response and current loop stage. User action controls call /loop/{id}/start or /skip; measure and feedback are saved through corresponding endpoints.

Demo flow

/demo is an authenticated demo console. It can seed one of two synthetic personas, trigger a sample capacity drop, time-warp an action window, check demo status, and link to the dashboard in demo context. Seeding resets records for the shared user_default demo profile and writes generated history. Treat this data as disposable/demo-only; do not use the demo feature with production data.

Routes and states

See [Frontend] for every route and expected auth requirement. General API failures are surfaced by the API wrapper as thrown errors; individual page components display their own loading/error/empty states. Exact visual state wording can change with UI code.

Authentication states

- Restoring: client checks /auth/me while auth provider initializes.

- Signed out: public auth pages remain available; protected data endpoints return 401.
- Signed in: provider exposes current user; profile-backed endpoints create/synchronize a SQLite profile row when needed.
- Session unavailable/expired: the API rejects the request; user must sign in again.

Empty and error behavior

New users can have no saved snapshots, actions, or history. API models provide a default capacity response when no snapshot exists and insights are computed from available records. Database/Mongo unavailability may cause API errors. Pages may show empty/error states rather than fabricated user activity. Demo seed data is synthetic and should be labeled as such.

System Architecture

Component diagram

```
flowchart LR
  U[User browser] --> W[Next.js / React frontend]
  W -->|credentialed HTTP JSON| A[FastAPI API]
  A -->|account + session lookups| M[(MongoDB)]
  A -->|signals, snapshots, interventions, outcomes, priors| S[(SQLModel
database: SQLite by default)]
  A --> E[Baseline / capacity / driver / drop / recommendation / measurement
engines]
  E --> D[actions.json + YAML weights and priors]
  E --> L[Explanation provider: template by default; optional providers]
  A --> W
```

Responsibilities

- Web: Next.js App Router routes and React components; forms, visualizations, session state, API calls.
- API: FastAPI routers perform validation, resolve identity, coordinate business logic, and serialize responses.
- Authentication store: MongoDB users and sessions collections. Mongo is contacted lazily by auth operations; health and non-auth requests can be available independently of it.
- Product store: SQLModel tables using DATABASE_URL, SQLite by default. Activity is stored server-side in the API process environment, not solely in the browser.
- Engine: Python modules compute baseline/capacity, drivers, drops, recommendations, and outcome summaries.
- Text layer: default deterministic template; optional Gemini/Ollama/OpenAI-compatible providers, with schema/phrase consistency validation and template fallback.

Request/data flow

The browser sends JSON and includes credentials. Authentication endpoints set or clear the HTTP-only session cookie. Protected requests hash the presented session token and validate the MongoDB session/user. The API maps the authenticated account ID to a product profile and scopes product queries by that ID. For a check-in it validates numeric ranges, intercepts configured crisis phrases, persists signals, computes a snapshot, and may persist an intervention. Later

requests update intervention status and create an outcome. Insights query those records.

Storage boundary

MongoDB is only the account/session store in the current code. SQLite (or another SQLModel-supported configured database) contains application profile/preferences and product data. The default SQLite URL is relative to the API working directory. Local .env path loading is also relative to process working directory; start Uvicorn from apps/api as documented.

Local development and deployment

Local defaults are frontend localhost:3000, API localhost:8000, MongoDB localhost:27017, SQLite file maxxloop.db. CORS permits the two localhost frontend origins on port 3000. Docker files exist, but the provided Compose file does not provide MongoDB and its web service configuration needs review for container-to-container API addressing and Next rewrite behavior. See [Deployment]; do not infer a verified production deployment.

Technical Architecture

Frontend

- Next.js 16.3.7 and React 19.3.0 according to apps/web/package.json.
- TypeScript, Tailwind CSS 3, Recharts, Lucide React, clsx, and tailwind-merge.
- App Router pages live under apps/web/src/app; shared components under src/components; request wrapper and browser consent storage under src/lib.
- Auth state uses a React context provider. The request helper uses fetch, JSON headers, and credentials: "include". NEXT_PUBLIC_API_URL selects the API base; default is http://localhost:8000.
- No separate global state framework is configured in the package manifest. Page/component React state is used.

Backend

- Python >=3.11; FastAPI, Uvicorn, Pydantic 2, SQLModel/SQLAlchemy.
- Routers are in apps/api/app/routers; schemas in models/schemas.py and models/auth_schemas.py.
- Engine modules under engine/ implement baselines, scoring, driver labels, drop rules, action selection, and measurement.
- Auth helpers use PyMongo, Argon2, secure random tokens, and FastAPI dependencies.
- Data/config files: actions.json, weights.yaml, population_priors.yaml.
- Optional numerical packages include NumPy, SciPy, pandas (declared dependency; inspect imports for actual usage before attributing a specific feature).

Data stores

SQLite via SQLModel stores User profile/preferences, Signal, CapacitySnapshot, Intervention, Outcome, and ActionPrior. MongoDB stores account and session documents. Indexes created in Mongo: unique normalized email, session expiry TTL, and session user ID. SQL fields have selected indexes on IDs/timestamps/kinds; no migration framework is present in the inspected repository.

Runtime configuration

Backend settings are Pydantic settings loaded from environment and .env (see [Environment]). The frontend has a NEXT_PUBLIC_API_URL variable. Next rewrites /api/* to http://localhost:8000/*; separately, current frontend code calls the API base directly. Verify actual deployment routing before using this rewrite as a production proxy.

Production status

The repository has an API Dockerfile and Compose file. No Vercel/Render deployment manifest was found in the inspected file inventory. Compose does not include the MongoDB service required for authentication. Production topology is therefore not established by these files alone.

Frontend

The Next.js App Router source is apps/web/src/app. Shared components are under src/components; API methods are centralized in src/lib/api.ts.

Route	Purpose / interaction	Auth
/	Public product landing and navigation	Public
/login	Email/password sign-in	Public
/signup	Create account	Public
/dashboard	Capacity, check-in, active action loop, follow-up	Authenticated data/API
/demo	Seed persona, trigger drop, time-warp demo	Authenticated API
/insights	Personal aggregates, trends, action summaries	Authenticated API
/history	Signal/history view	Authenticated API
/profile	Profile and preference surface	Auth-related/product APIs as implemented
/onboarding	Initial setup/preferences flow	Product route; API requirements depend on page actions
/privacy	Consent, payload inspection, export, delete controls	Authenticated API

These route files exist in the source tree. This list is not a claim that a browser test was run during documentation generation.

Important components

AuthProvider restores current user state; AuthLayout frames auth pages; Navbar links product pages; CapacityGauge and SparklineChart visualize status/history; LoopRing and CycleProgress communicate stages; DriverChip and WhyThisDrawer show context; ActionTimer supports timed actions; DemoBar exposes demo controls; CrisisModal presents the crisis-intercept result; ThemeProvider controls theme state; StatusMessage renders status feedback.

API communication

The API wrapper at src/lib/api.ts uses NEXT_PUBLIC_API_URL or http://localhost:8000, JSON requests, and browser credentials. It exposes auth, check-in, capacity, loop, insights, signals, privacy, and demo operations. Non-2xx responses become JavaScript errors using API detail where available. Per-page components own loading and error presentation.

Styling and responsiveness

Tailwind and global CSS define styles; responsive behavior should be confirmed at the target browser sizes. No automated browser/responsive test configuration was found in the inspected repository. Refer to [Testing].

Backend

Application

apps/api/app/main.py creates the FastAPI application, initializes SQLModel tables in lifespan, configures credentialed CORS for http://localhost:3000 and http://127.0.0.1:3000, mounts domain routers, and exposes GET /health. Swagger UI is /docs; ReDoc is /redoc.

Routers and processing

- auth: signup/login/logout/current account; MongoDB-backed account/session storage.
- signals: create, batch-create, list numeric signals; calendar/browser-source values respect profile consent booleans.
- checkins: scan note, save self-report signals, compute capacity/baseline/drivers/drop, optionally create action and explanation.
- capacity: latest capacity response and recent sparkline.
- loop: active loop, start/skip, measure, feedback.
- insights: aggregate saved outcomes, drivers, timestamps, and trends.
- privacy: inspect in-memory latest provider transaction, export product records, delete product records, update consent/preferences.
- demo: authenticated seed, trigger drop, timewarp, and status controls.

Engine services

BaselineEngine, CapacityEngine, DriverEngine, DropDetectionEngine, RecommenderEngine, and MeasurementEngine are called from the route handlers. Validation is mostly Pydantic request/response models. SQL operations use injected SQLModel sessions. Auth uses FastAPI dependencies to resolve the session and profile.

Request lifecycle example

POST /checkins -> validate values -> crisis scan optional note -> persist focus/energy/stress signals -> load latest ancillary signals -> compute score/baseline/drivers -> evaluate drop and cooldown -> if eligible, sample an action and generate explanation -> save snapshot/intervention -> return response. See [MaxxLoop loop].

Errors and operations

FastAPI/Pydantic emits standard validation errors; auth uses 401 for invalid credentials/unauthenticated access and 503 when Mongo auth storage is unavailable. Duplicate email signup returns 409. Database operational errors may surface according to route and driver behavior. Health reports process/config status, not a full readiness check for MongoDB or SQLite.

API Reference

Base URL in local development: http://localhost:8000. OpenAPI/Swagger: /docs; ReDoc: /redoc. Requests that need identity use the HTTP-only session cookie. JSON examples below are based on the Pydantic schemas and route implementation; actual

responses include generated IDs/timestamps and may differ by data.

System and authentication

Method/path	Auth	Purpose / effect
GET /health	No	API process/config status. Does not prove Mongo is reachable.
POST /auth/signup	No	Body {name,email,password}; creates Mongo account/session, sets cookie; 201, duplicate email 409, Mongo unavailable 503.
POST /auth/login	No	Body {email,password}; sets session cookie; invalid credentials 401.
POST /auth/logout	No	Invalidates current session if provided, clears cookie.
GET /auth/me	Yes	Returns {user:{id,name,email,createdAt}}.

Product endpoints

Method/path	Auth	Purpose / effect
POST /signals	Yes	Body {kind,value,source?,ts?}; stores one numeric signal unless source consent disallows it.
POST /signals/batch	Yes	Body {signals:[{kind,value,source?,ts?}]}; stores allowed signals, returns saved_count.
GET /signals?kind=&limit=	Yes	Current user's signals, newest first; default limit 50.
POST /checkins	Yes	Body defined below; records a check-in and may create a snapshot/intervention, or returns crisis interception.
GET /capacity/now?demo=false	Yes	Latest capacity, drivers, status and 14-day sparkline; demo flag selects demo profile behavior.
GET /loop/active?demo=false	Yes	Current active loop/stage/action/timing/outcome context.
POST /loop/{intervention_id}/start?demo=false	Yes	Marks offered intervention started.
POST /loop/{intervention_id}/skip?demo=false	Yes	Body {reason?}; marks intervention skipped.
POST /loop/{intervention_id}/measure?demo=false	Yes	Body {focus_self,energy_self,stress_self,note?}; saves signals/snapshot/outcome, computes measurement and updates prior.
POST /loop/{intervention_id}/feedback?demo=false	Yes	Body {helpful,note?}; records user feedback for outcome when available.
GET /insights?demo=false	Yes	Aggregates closed loops, helpfulness, improvements, actions, recurring drivers, time distribution, weekly trend.

‘POST /checkins‘ request

```
{
  "focus_self": 3.0,
  "energy_self": 3.0,
  "stress_self": 2.0,
  "note": "Optional text",
  "tags": [],
  "window_context": "normal"
}
```

The three numeric values must each be from 1 to 5. The response normally contains

status, snapshot_id, score, baseline, delta, is_drop, drop_reason, cooldown_ok, drivers, and nullable active_intervention. If the safety phrase scanner intercepts the note, the response instead has status: "crisis_intercepted", a message, and crisis_card; normal persistence/recommendation is paused.

Privacy and demo endpoints

Method/path	Auth	Purpose / effect
GET /privacy/llm-payload	Yes	Returns last in-memory provider transaction for that user (not durable audit history).
GET /privacy/export	Yes	Returns the user profile and product signal/snapshot/intervention/outcome records.
DELETE /privacy/data	Yes	Deletes the user’s product records and priors; keeps account/profile and changes consent flags.
POST /privacy/consent?...	Yes	Query args consent_calendar, consent_browser_signals, consent_llm_sharing, optional llm_provider; updates SQL profile.
POST /demo/seed	Yes	Body {persona:"aarav" | "meera"}; resets and writes shared synthetic user_default data.
POST /demo/trigger-drop	Yes	Optional {drop_severity} body; generates sample demo signals/snapshot/action.
POST /demo/timewarp	Yes	Optional {seconds} body; advances eligible demo intervention timing.
GET /demo/status	Yes	Reports demo mode/provider/persona and configured timewarp information.

Schema constraints and errors

SignalCreate: required string kind, float value, optional source default self_report, optional timestamp. CheckinCreate and loop measurement values are 1-5. Signup email uses EmailStr, password limits 8-128. Common HTTP failures include 401 unauthenticated/invalid credentials, 409 duplicate signup, 422 validation errors, and 503 unavailable Mongo authentication storage. Specific route details are available in runtime OpenAPI.

Example authenticated loop

```
POST /checkins
GET  /loop/active
POST /loop/{id}/start   (or /skip)
POST /loop/{id}/measure
POST /loop/{id}/feedback
GET  /insights
```

There is no /api prefix in FastAPI routes. Frontend requests use the configured base URL directly. See the request wrapper in apps/web/src/lib/api.ts.

Database

Two-store arrangement

Store	Purpose	Configuration
SQLModel relational DB (SQLite by default)	Product profile/preferences and signals, snapshots, interventions, outcomes, action priors	DATABASE_URL
MongoDB	Authentication accounts and server-side sessions	MONGODB_URI,

MONGODB_DATABASE |

Default SQLite URL: sqlite:///./maxxloop.db. The relative file location depends on API process working directory; standard instructions launch from apps/api.

SQL tables

See [Data model] for fields. SQLModel creates tables at app startup via SQLModel.metadata.create_all; no schema migration tool was found. Some columns store JSON strings (drivers_json, explanation_json) and are converted by model properties or route code.

Mongo collections and indexes

- users: generated _id, name, normalized email, passwordHash, creation/update timestamps. Unique index on email.
- sessions: _id is SHA-256 hash of opaque cookie token, userId, createdAt, expiresAt. TTL index on expiresAt, plus userId index.

The Mongo client connects lazily on auth operations. TTL cleanup is managed by MongoDB TTL behavior; request authentication also checks expiry.

User scoping

Protected dependencies map the authenticated Mongo user ID to a SQL User profile. Product routers generally filter records by that ID. Outcomes are linked to an intervention, which is scoped to a user when the intervention is retrieved/queried. Demo endpoints intentionally operate on the shared user_default synthetic profile and reset its product records; do not use this feature as normal user storage.

Lifecycle and limitations

SQLite data is server-side wherever the API runs. A persistent volume and backups are needed for a container deployment. Deleting /privacy/data removes the user's product records and action priors while preserving the account and disabling consent flags; it does not delete the Mongo account/session. Account deletion is not exposed by this endpoint.

Data Model

SQLModel entities

Entity	Key fields	Purpose / relationship
User	id, display_name, timezone, created time, consent flags, LLM provider	SQL profile keyed by authenticated Mongo user ID; default model ID exists for demo compatibility.
Signal	integer ID, user_id, timestamp, kind, numeric value, source	Individual observation; user and timestamp/kind are indexed.
CapacitySnapshot	ID, user_id, timestamp, score, baseline, delta, driver JSON, drop flag, context	Calculated state at a moment.
Intervention	ID, user_id, snapshot_id, action ID, chosen time, window, status, explanation JSON, provider, exploration flag	One offered action tied to a snapshot. Status values documented in model: offered/started/completed/skipped.
Outcome	ID, intervention_id, pre/post score, observed delta, expected no-action delta, net effect, confidence, helpfulness, note	Follow-up measurement and optional

user feedback for an intervention. |
| ActionPrior | ID, user_id, action_id, alpha, beta, updated time | Recommender state for a user/action pair. |

There is no dedicated SQL Loop, CheckIn, Action, or Insight table. Check-ins are represented as signals plus snapshots; actions are in actions.json; insights are computed from records.

Entity links

```
erDiagram
  USER ||--o{ SIGNAL : owns
  USER ||--o{ CAPACITY_SNAPSHOT : owns
  CAPACITY_SNAPSHOT ||--o{ INTERVENTION : context_for
  USER ||--o{ INTERVENTION : owns
  INTERVENTION ||--o{ OUTCOME : measured_by
  USER ||--o{ ACTION_PRIOR : has
```

These are logical relationships; some are represented by integer/string fields rather than declared ORM Relationship constraints. Mongo users and sessions are separate from the SQL tables, joined by user ID in application code.

Data lifecycle

Check-ins add signals and snapshots. Recommendation creates an intervention and may initialize priors. Measurement creates an outcome and updates an existing action prior. Export returns product records. Delete-data removes the user's signals/snapshots/interventions/outcomes/priors but retains SQL profile and Mongo account.

Authentication

Signup

POST /auth/signup accepts name, valid email, and password length 8-128. The schema trims the name and lowercases/normalizes email. The auth helper hashes passwords using Argon2 and inserts a MongoDB user. A unique email index prevents duplicate accounts (409). Successful signup immediately creates a session and sets the session cookie.

Login and session

POST /auth/login accepts email/password and verifies the stored Argon2 hash. Unknown emails and mismatched passwords return the same 401 message. A random opaque token is generated; its SHA-256 hash is stored in MongoDB. The raw token is set as an HTTP-only, SameSite=Lax cookie, with seven-day max age. The secure attribute is enabled when APP_ENV is not development/dev/local/test. A Mongo TTL index supports expiry cleanup, and requests also check expiry.

Logout/current user

POST /auth/logout deletes the current server-side session when present and clears the cookie. GET /auth/me resolves the session and returns public user fields. Product endpoints use FastAPI auth dependencies and return 401 if unauthenticated. A SQL profile keyed by authenticated user ID is created/synchronized on demand.

Authorization and limitations

Product queries use the current user ID; demo APIs are a special case that

manipulate a shared synthetic user_default dataset. Do not treat demo operations as a production multi-user demonstration. Auth account data and product records live in different databases. This describes implementation, not a full security audit; production needs HTTPS, secret management, deployment configuration review, monitoring, and security testing.

The MaxxLoop Loop

Stage 1 - Input and check-in

The user reports Focus, Energy, Stress on 1-5 sliders and may add a note. The web client sends POST /checkins. CheckinCreate validates values. The safety scanner runs before the normal save path; detected phrases return a crisis card and pause processing.

Stage 2 - Signal processing and snapshot

For a normal check-in, the API stores three Signal rows with source self_report, queries the latest values for supported additional kinds, and calls BaselineEngine and CapacityEngine. CapacitySnapshot stores score, baseline, delta, drivers JSON, drop flag, context, user ID, and timestamp. This is a server-side SQL record.

Stage 3 - Drivers and drop decision

DriverEngine turns contributions into labels and selects up to three. DropDetectionEngine evaluates configured absolute/rapid thresholds, cooldown and daily limits. The capacity result is an algorithmic estimate, not a clinical measurement. If no eligible drop exists, the user still sees the capacity/status without a new intervention.

Stage 4 - Recommendation and explanation

When eligible, RecommenderEngine filters the action library by driver overlap, initializes user/action Beta priors as needed, samples candidate values, and selects one action (with a default 15% random-candidate exploration branch). The text layer builds a structured explanation using the user’s selected provider or template fallback. An Intervention stores action/status/window/explanation/provider and exploration metadata.

Stage 5 - Action

The client shows the action and timer/steps. Start calls POST /loop/{id}/start; skip calls /skip. These update status and create data later used for measurement/control-window estimation. The standard window is configured for 25 minutes; demo timewarp can shorten waiting.

Stage 6 - Measurement

The user reports focus/energy/stress again via /measure. The API computes a new score and stores signals/snapshot. MeasurementEngine calculates observed_delta = post - pre, estimates no-action drift using subsequent snapshots after skipped interventions (blending a default prior when history is sparse), and calculates net_effect = observed_delta - expected_delta_no_action. It stores an Outcome and confidence category. This is observational self-report, not causal proof.

Stage 7 - Feedback and update

POST /feedback can attach helpfulness to the outcome. Measurement updates the

matching action prior: alpha increments when net effect >2; beta otherwise. Helpful boolean is recorded separately; it is not the criterion for this prior update in current code.

Stage 8 - Insights and next loop

GET /insights aggregates saved outcomes, action use, helpfulness, driver categories, time distribution, and trends. Later check-ins can use accumulated history and priors. Sparse history uses population priors/fallback values. The data model stores linked records, not a standalone Loop row.

```
sequenceDiagram
  actor User
  participant Web as Next.js
  participant API as FastAPI
  participant Engine as Engines
  participant SQL as SQLite/SQLModel
  participant Mongo as MongoDB auth
  User->>Web: Submit check-in
  Web->>API: POST /checkins (session cookie)
  API->>Mongo: Resolve account/session
  API->>Engine: Score, baseline, drivers, drop
  Engine->>SQL: Save signals and snapshot
  API->>Engine: Select action and explain
  Engine->>SQL: Save intervention/prior
  API-->>Web: Capacity and optional action
  User->>Web: Start/skip, then follow-up
  Web->>API: /loop/{id}/measure and /feedback
  API->>Engine: Calculate outcome/update prior
  Engine->>SQL: Save outcome
  Web->>API: GET /insights
```

Recommendation Engine

What it is

The recommender is a statistical selection mechanism over a curated JSON action library. It is not a trained neural network or general-purpose AI model. The language provider creates explanatory text and is separate from action selection.

Inputs and candidate filtering

The engine receives the user ID and top driver kinds from the capacity/check-in path. It loads apps/api/app/data/actions.json (24 action records in the current file). Candidate actions are those whose target_drivers overlap active drivers. If no overlap exists, it falls back to selected general restorative categories.

Selection

Each (user_id, action_id) has an ActionPrior with alpha and beta. A new prior is initialized from action prior_effect using total strength 4: alpha = effect x 4, beta = (1-effect) x 4. The engine samples theta ~ Beta(alpha,beta) per candidate and usually selects the highest draw. With default probability 0.15 it chooses a random candidate as exploration when there is more than one. This is Thompson-sampling-style selection with an explicit exploration branch.

Outcome update

At measurement, the engine calculates observed post-minus-pre score, subtracts estimated no-action drift, and labels confidence from sample count/variance. If net effect is greater than 2 points, alpha increments; otherwise beta increments. User helpfulness is separately recorded. The current code's prior update is based on net effect threshold, not solely on the explicit helpful/not-helpful boolean.

Interpretation and limits

Self-report changes, estimated drift, action selection, and prior updates are not randomized evidence. A positive net score does not prove the action caused improvement. Synthetic simulation is not a user study. No validated ML model, online training infrastructure, or causal inference design is implemented.

Insights

GET /insights builds a response from stored snapshots, interventions, outcomes, and drivers for the authenticated user (or demo profile when explicitly selected).

The response schema includes:

- total_loops_closed and current_streak
- pct_helpful from recorded helpfulness responses
- avg_score_improvement based on stored measurement outcomes
- top_actions with action ID/title/category, use count, mean net effect, interval bounds, and helpful ratio
- recurring drivers, time-of-day distribution, weekly trend

These are descriptive aggregates of available records. A stored net_effect subtracts an estimated no-action drift; it is not a measured causal treatment effect. Action confidence labels/interval calculations should be interpreted as prototype estimates. Sparse data can produce empty or low-information results. Demo seed data includes synthetic records and should be labeled accordingly.

The frontend's /insights page renders the API response and charts/components. /history calls the signals endpoint with a limit. Exact charts and empty-state copy should be read from the current route files.

Security

This document records observable implementation safeguards and gaps; it is not a security certification.

Implemented

- Argon2 password hashing; plaintext passwords are not persisted by the auth helper.
- Random opaque session token; Mongo stores its SHA-256 hash, with expiry and per-user indexes.
- HTTP-only, SameSite=Lax cookie, seven-day max age; Secure set outside local/test environments.
- Protected API dependencies resolve sessions and use authenticated user IDs for product queries.
- Pydantic validates key request fields and numeric ranges.
- Unique normalized email index; duplicate account returns 409.
- LLM provider output is parsed into a schema and checked against banned phrases/driver consistency; provider exceptions fall back to template.

Configuration and limits

- CORS currently allows only localhost:3000 and 127.0.0.1:3000, with credentials. Production origins need explicit configuration/code review.
- No rate limiting, account lockout, password reset, email verification, or account deletion endpoint was found in the inspected implementation.
- CSRF protections beyond SameSite cookie behavior were not identified. Production cookie/domain/CORS and CSRF threat model require review.
- Do not expose .env, API keys, MongoDB credentials, or user exports. Keep secrets in the deployment's secret manager.
- SQL migrations, security monitoring, backup encryption, vulnerability scanning, and external security audit are not established by repository code.
- The shared demo profile has destructive reset behavior; keep demo operations away from production data.

Production checklist

Use HTTPS, restrict CORS to the real frontend origin, use managed secret storage and private database networking, configure durable backups and access controls, review authentication/session/CSRF/rate-limit behavior, and verify deletion/retention requirements before public use.

Privacy and Data Handling

Data currently handled

- Account name/email, password hash, and session records are stored in MongoDB.
- Product profile preferences, numeric signals, capacity snapshots, driver labels, intervention/action status, outcomes, and optional measurement note are stored in the API's SQL database (SQLite by default).
- Check-in note text is scanned for configured crisis phrases before the standard check-in persistence path. On a detected phrase, that route returns a crisis card without saving the check-in note/signals. This is limited phrase matching, not comprehensive detection.
- The default template provider is deterministic and requires no third-party call. Optional providers can send structured score/baseline/driver/action data to configured model services. Provider-specific data flows must be disclosed when enabled.

Consent and user controls

Profile fields include calendar/browser/LLM consent flags; signal ingestion skips calendar/browser source data when their consent is false. /privacy/consent changes those fields and the selected provider. The UI/API provide payload inspection, data export, and product-data deletion. Deletion retains account/session infrastructure and SQL profile while deleting product records and disabling consent flags.

The LLM payload inspector is process-memory state, not a durable per-user audit log. Its exclusion list is application metadata; do not interpret it as an independent network inspection or guarantee.

Accurate claims

Activity data is not device-only: it is persisted by the backend. Do not claim zero-knowledge, GDPR/DPDP compliance, or that no information leaves the server. No privacy certification/legal review is documented. Production operators must define retention, backups, access, and deletion behavior for both Mongo and SQL stores.

Testing

Existing automated backend tests

The repository contains pytest modules:

- test_auth.py - authentication behavior
- test_e2e_loop.py - end-to-end API loop behavior
- test_engine.py - capacity/baseline/driver engine behavior
- test_measurement.py - outcome measurement behavior
- test_recommender.py - action selection behavior
- test_safety.py - safety phrase behavior

Run from apps/api:

```
pytest -v tests/
```

or from the repository root: make test.

Frontend checks

The web package scripts are dev, build, start, and lint. It declares a Next build and a lint script (next lint); verify the lint command against the installed Next version because script compatibility can change. TypeScript is configured with tsconfig.json, but no dedicated test runner script or browser automation configuration was found in the inspected files.

Coverage boundaries

No frontend unit test, browser automation, or responsive visual regression configuration was found. Backend tests do not alone demonstrate production database behavior, accessibility, or cross-browser responsiveness.

Tests were not executed during documentation generation. This documentation change has not been verified by running the app/build/test suites.

Local Development

Prerequisites

- Git
- Node.js and npm (lockfile is apps/web/package-lock.json)
- Python 3.11 or newer and pip
- MongoDB running/reachable for signup, login, and protected authenticated requests

SQLite is the default product database and is created by SQLModel on API startup. No separate SQLite server is required.

Clone and configure

```
git clone https://github.com/Kingtheblaze/MaxxLOOP.git
cd MaxxLOOP
```

Create backend environment file from .env.example at apps/api/.env; edit Mongo URI if needed. Do not commit real credentials. The settings loader reads .env relative to the current process directory, so run the API from apps/api.

Install

PowerShell:

```
cd apps/api
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item ..\..\.env.example .env
```

Bash equivalent for venv activation: source .venv/bin/activate. Then install web dependencies:

```
cd apps/web
npm ci
```

Run in two terminals

Terminal 1, from apps/api with the virtual environment active:

```
uvicorn app.main:app --reload --port 8000
```

Terminal 2, from apps/web:

```
npm run dev
```

Open http://localhost:3000. API docs: http://localhost:8000/docs; health: http://localhost:8000/health. Authentication requires MongoDB; the health route does not check Mongo readiness.

Other commands

- Backend tests: cd apps/api && pytest -v tests/.
- Web production build: cd apps/web && npm run build.
- Docker Compose: docker compose up --build; see [Deployment] because Compose currently omits MongoDB and has container URL caveats.
- make dev prints instructions; it does not run two servers concurrently.

Environment Variables

Backend settings are defined in apps/api/app/core/config.py; .env.example is the safe template. Never copy real secrets into documentation or source control.

Variable	Required?	Default/example	Purpose
APP_ENV	No	development	Cookie Secure behavior and environment label.
PORT	No	8000	Settings value; documented Uvicorn command explicitly supplies port.
DEBUG	No	true	Settings flag; inspect consumers before relying on behavior.
DATABASE_URL	No	sqlite:///./maxxloop.db	SQLModel product database.
MONGODB_URI	Required for auth use	mongodb://localhost:27017/	Mongo connection string; treat as secret if credentials included.
MONGODB_DATABASE	No	maxxloop	Mongo database name.
SESSION_COOKIE_NAME	No	maxxloop_session	Session cookie key.
LLM_PROVIDER	No	template	template, gemini, ollama, or openai; unavailable/keyless providers fall back to templates.
GEMINI_API_KEY	Only for Gemini	unset	Gemini credential.
OLLAMA_BASE_URL	Only for Ollama	http://localhost:11434	Ollama service endpoint.
OLLAMA_MODEL	Only for Ollama	llama3.2	Ollama model identifier.
OPENAI_API_KEY	Only for OpenAI-compatible	unset	Provider credential.

OPENAI_BASE_URL	No	https://api.openai.com/v1	Compatible API base URL.
OPENAI_MODEL	No	gpt-4o-mini	Requested model identifier.
DEMO_MODE	No	true	Demo status/config flag.
TIMEWARP_SECONDS	No	20	Demo timewarp duration.
STANDARD_WINDOW_MINUTES	No	25	Standard action measurement window.
DROP_THRESHOLD_POINTS	No	10.0	Drop threshold setting.
RAPID_DROP_POINTS	No	15.0	Rapid drop setting.
MIN_COOLDOWN_MINUTES	No	45	Intervention cooldown setting.
MAX_INTERVENTIONS_PER_DAY	No	3	Daily intervention cap.
NEXT_PUBLIC_API_URL	No	http://localhost:8000	Frontend browser API base, set in web environment.

Backend variables are loaded by Pydantic settings. The web client in apps/web/src/lib/api.ts reads NEXT_PUBLIC_API_URL at build/runtime according to Next.js public env behavior. Do not set public variables to credentials.

Deployment

Current repository configuration

An API Dockerfile and docker-compose.yml are present. The Compose file exposes API port 8000 and web port 3000, uses SQLite and template provider defaults, and mounts an API data volume. It does not define MongoDB even though auth requires MongoDB. The web container sets NEXT_PUBLIC_API_URL=http://localhost:8000; inside a browser this means the user's machine, and inside containers service DNS differs. The Next rewrite is also hard-coded to localhost:8000. Therefore do not assume the supplied Compose file is a working public deployment without testing and configuration changes.

No verified Vercel/Render/Atlas deployment or provider manifest was found in the repository inventory. This document gives deployment requirements, not a claim deployment exists.

Production requirements

1. Deploy the web app and API to reachable HTTPS origins.
2. Provide MongoDB reachable only by the API; configure MONGODB_URI and MONGODB_DATABASE using a secret store.
3. Configure DATABASE_URL for durable product data or mount the SQLite file on a persistent volume. SQLite on ephemeral serverless filesystems is not durable.
4. Set APP_ENV=production so auth cookies use Secure; use HTTPS and configure correct cookie domain/SameSite behavior for the chosen topology.
5. Set the frontend API URL to the public API origin. Update API CORS allowlist to the exact frontend origin and verify credentialed cookie behavior.
6. Configure LLM provider credentials only if using that provider; review data disclosures/consent.
7. Configure backups, logs/monitoring, database access, privacy retention, and deletion procedures.

Build/run references

- Web: cd apps/web && npm ci && npm run build; serving: npm run start (package script binds port 3000).
- API: install apps/api/requirements.txt; start with uvicorn app.main:app --host 0.0.0.0 --port 8000.
- API Docker image: build from apps/api context using its Dockerfile.

Do not publish until the production URL, auth cookies, CORS, Mongo connectivity, durable SQL storage, and data isolation are verified in the target hosting environment.

Troubleshooting

Frontend cannot connect to API

Check the API is listening on the URL in NEXT_PUBLIC_API_URL (default http://localhost:8000) and /health responds. Browser CORS currently allows only localhost:3000 and 127.0.0.1:3000. Restart Next after changing environment variables. The API wrapper calls the configured base directly.

MongoDB connection failure / auth unavailable

Start MongoDB, confirm MONGODB_URI/MONGODB_DATABASE in apps/api/.env, then restart Uvicorn from apps/api. Auth routes connect lazily and return 503 when Mongo is unavailable; /health can still report healthy because it does not ping Mongo.

CORS or session cookie issue

Use matching localhost hostnames (avoid mixing localhost and 127.0.0.1 without considering cookie origin). For a remote frontend, update the API CORS allowlist in main.py, keep credentials enabled, use HTTPS in production, and verify cookie Secure/SameSite/domain settings.

Invalid login / duplicate account

Emails are normalized to lowercase. Signup with an existing email returns 409. Login returns the same 401 for missing email and wrong password. Password minimum on signup is 8 characters.

API 401

The session cookie may be missing, expired, or invalid. Sign in again and verify browser requests include credentials. Protected API routes require a valid session.

API 422 / 500

422 usually means a request body or query argument does not match the Pydantic schema/range; check /docs. For 500, inspect Uvicorn logs and database availability. Avoid sharing logs containing personal data or secrets.

Environment variable appears ignored

The backend reads .env relative to process working directory; run Uvicorn from apps/api. Restart server after edits. NEXT_PUBLIC_API_URL is read by the Next app; restart/rebuild as needed.

Port already in use

Choose a free API port and keep frontend URL, Next rewrite, and CORS configuration aligned. Current defaults are 8000 and 3000. Do not assume changing Uvicorn’s port updates the frontend automatically.

Production build failure

Run npm ci then npm run build in apps/web; read the first compiler error and

confirm compatible Node.js. For API import/dependency errors, activate the intended Python environment and install requirements.txt. Build/test status should be reported only after actually running commands.

Docker auth does not work

The supplied Compose file has no MongoDB service. Its API container cannot reach host Mongo at localhost as though it were inside the host. Configure a reachable Mongo service/URI and review inter-container web/API addresses before using Compose.

MaxxLoop Hackathon Guide

This is the current, implementation-grounded guide to explaining and demonstrating MaxxLoop. Use this document over older pitch notes when they make stronger claims than the running code supports. In particular, do not describe MaxxLoop as a doctor, clinically validated, zero-knowledge, or already deployed to production.

1. MaxxLoop in one sentence

MaxxLoop helps students and knowledge workers notice when their focus and energy are slipping, choose one small recovery action, and check whether it helped them personally.

The product loop is Track -> Understand -> Act -> Measure -> Improve. The main idea is to make a wellness check-in lead to a useful next step and a follow-up measurement, instead of stopping at a chart or a generic list of advice.

30-second pitch

"Students and knowledge workers often realize they are overloaded only after their focus has already dropped. Most tools show historical charts or offer generic tips. MaxxLoop uses lightweight self-reported signals to estimate a person’s current capacity against their own recent baseline. It explains the main contributors, suggests one small action, and asks for a follow-up check-in to learn what seems helpful for that person. It is a wellness-support prototype, not a medical device or a substitute for professional care."

2. Problem, users, and solution

The problem

People have limited attention and energy, especially during study or demanding work. A dashboard may show a low score without telling someone what to do next. A long list of wellness tips can also make a tired person choose among too many options. There is room for a simple experience that connects a check-in, an understandable explanation, one action, and a follow-up.

Who it is for

The initial audience is students and knowledge workers who want lightweight support for focus and recovery. The current product does not diagnose burnout, anxiety, depression, ADHD, or any other condition.

What the app does today

1. The user signs in and completes a short check-in for focus, energy, and stress, with an optional note.
2. The API compares recent signals with the user’s available baseline and

estimates a capacity score.
3. The app presents contributing factors in plain language and recommends a single small action.
4. The user can start or skip the action, then record another check-in after the action window.
5. The user can say whether the action felt helpful. The app records the outcome and uses past results to inform later recommendations.
6. Insights and history show the user’s recorded activity and outcomes.

The current demo also has seeded personas and controls to create a sample capacity drop or advance a measurement window. Demo data is illustrative and may not produce identical scores every time.

3. What makes it different

Describe this as MaxxLoop’s product distinction, not as a claim that no other product does something similar:

- It closes the loop. The experience connects a check-in to an action and a later measurement, instead of only collecting data or displaying a trend.
- It starts with the individual’s baseline. When enough history exists, the scoring logic compares signals with that user’s own recent pattern; sparse history uses prior values until more observations exist.
- It gives one next action. The product aims to reduce decision effort when the user reports low capacity.
- It makes the reasoning inspectable. The app can show contributing signal categories and recommendation context rather than presenting a mysterious score alone.
- It learns from feedback. A lightweight Beta/Thompson-sampling recommender balances actions that have worked in prior observations with some exploration. This is an engineering approach in a prototype, not clinical proof that an action works.
- It has a safety interruption. Certain crisis-related phrases in check-in notes are intercepted and routed toward support information before the usual recommendation flow. This keyword scanner is limited and cannot guarantee that a crisis is detected.

Simple comparison for judges

Typical experience	MaxxLoop’s intended contribution
A chart reports a trend	A check-in leads to an explanation and a next step
A list shows many tips	The app offers one small action at a time
Advice is not followed up	A later check-in records the user’s observed response
Generic suggestions	Recommendations can adapt to the user’s recorded feedback

Avoid naming competitors or saying "first," "only," "proven," or "guaranteed" unless you have independent evidence.

4. Fit with the hackathon

The official ASYNC’26 Wellness & Lifestyle problem statement calls for turning everyday signals into understanding and improvement, and describes a flow of Track -> Understand -> Act -> Measure -> Improve. That aligns closely with MaxxLoop’s product loop. The published idea-deck guidance asks for the project/team, track, problem, idea, solution, technical architecture/workflow, technologies, and

differentiator. Its published idea-stage scoring weights are technical execution/architecture 30%, innovation 20%, impact 20%, product experience/usability 15%, and demo/completeness 15%.

The event page also lists the final build period as September 30-October 1, 2026. Treat the published weights as the rubric on that page; do not assume they necessarily constitute a separate final-round judging rubric unless the organizers say so.

Sources: [ASYNC'26 problem statements and idea-deck criteria] • [ASYNC'26 rules]

How to map your explanation to the published criteria

- Technical execution / architecture: show the real frontend-to-API flow, auth boundary, scoring, recommendation, and measurement.
- Innovation: explain the complete personal feedback loop and one-action experience. Frame it as your design contribution, not a universal first.
- Impact: explain the focused initial audience and why a timely, small action could help. Present this as a hypothesis to validate with users.
- Product experience: demonstrate a concise check-in, understandable explanation, one recommendation, and follow-up.
- Demo / completeness: finish one full loop and show its record in history or insights. Be transparent about seeded demo controls and the limits of the current evidence.

5. Technical overview

Stack

- Web: Next.js App Router, React, TypeScript, Tailwind CSS, Recharts, and Lucide icons.
- API: Python FastAPI, Pydantic, SQLModel/SQLAlchemy.
- Product records: SQLite stores signals, check-ins, capacity snapshots, interventions, outcomes, and related consent/profile data through the configured DATABASE_URL (local default: sqlite:///./maxxloop.db).
- Authentication: MongoDB stores account and server-side session records, configured with MONGODB_URI and MONGODB_DATABASE. In the local setup, the URI points to the developer's MongoDB service. Do not publish connection strings or credentials.
- Language layer: a deterministic template provider works without an external API key. Optional provider integrations exist in the code; do not claim a remote model is active unless you have configured and verified it for the demo.

Architecture at a glance

```
Browser (Next.js / React)
       ? HTTP requests, session cookie
       ?
FastAPI application
  ??? Authentication and session checks ?? MongoDB
  ??? Check-ins, scoring, loops, insights ?? SQLite
  ??? Recommendation and measurement logic
  ??? Optional language provider (template by default)
```

Main request flow

1. The browser sends a request to the FastAPI service; the frontend's API URL is controlled by its configuration/environment.

2. Protected API routes resolve the authenticated user from the session cookie.
3. User-owned product records are read or written with that user identity in scope.
4. The check-in is evaluated, a capacity estimate and contributing factors are computed, and a candidate action is selected.
5. Starting/skipping, measuring, and helpfulness feedback update the loop record and recommendation history.
6. Insights and history read back the user's stored records.

The repository includes endpoints grouped around /health, /auth/*, /signals, /checkins, /capacity/now, /loop/*, /insights, /privacy/*, and /demo/*. The API docs and route implementation are the source of truth for request schemas.

How to explain the scoring

Use a simple explanation first: "The score is an estimate of current capacity relative to recent patterns, not a medical measurement."

For a technical judge:

- The system uses recent personal history where enough observations exist and falls back or blends with broader prior values where history is sparse.
- It uses robust statistics (including a median and median absolute deviation) to reduce the influence of unusual observations.
- Available signals are standardized, direction-adjusted where needed, weighted, and combined into a bounded 0-100 estimate. A score near 50 is the reference point in this scoring scheme, not a population norm or clinical threshold.
- A drop may open a recommendation loop. Cooldowns and daily limits constrain repeated interventions.

Do not call this a clinically validated biomarker or say a score diagnoses a person's state.

How to explain personalization

"We keep lightweight success/failure evidence for actions and use Thompson sampling over Beta distributions to choose between actions with promising history and some exploration. It is a way to personalize recommendations from observed app feedback. Our current simulation is synthetic, so we still need real-user evaluation before making effectiveness claims."

The measurement compares the user's before and after self-report and estimates expected no-action drift from prior control-like windows. Some values fall back to a prior when there is little history. This is not a randomized controlled trial and does not establish that the action caused the change.

6. Privacy, authentication, and data

Be precise about where information currently lives:

- MongoDB is used for authentication accounts and sessions.
- SQLite is used by the API for product activity and recommendation records. In a deployed setup, that would be server-side storage with a persistent volume; it is not device-only storage.
- Passwords are stored as password hashes, not plaintext. Session cookies are HTTP-only; server-side session tokens are stored in hashed form.
- The privacy UI includes consent, export, and deletion-related controls. Explain what these do only after confirming the current deployed environment and retention behavior.

- The default language provider is a local deterministic template implementation. If an external LLM provider is enabled, explain its configured data flow and consent accurately. Do not promise that no information ever leaves the device.
- The app's optional note is sensitive. The safety scanner intercepts recognized crisis phrases before the normal flow; the scanner is not a complete safety monitor.

Do not repeat older repository pitch claims that MaxxLoop is "zero-knowledge," that all data stays on the user's device, or that the app is GDPR/DPDP compliant. Those are not established by the current architecture or this guide. Also do not expose .env contents, passwords, MongoDB credentials, or real user records during a demo.

7. The doctor / clinical part

There is no doctor or clinician built into the current product, and no clinical review or validation is established by the current project materials. MaxxLoop is a non-medical focus and wellness-support prototype. It does not diagnose, treat, prescribe, or replace a clinician.

The responsible answer to "What does the doctor do?" is:

> "A doctor is not part of the current product, and we are not presenting this as medical care. The current safety behavior is a limited phrase-based interruption that displays support resources. Before using this with real users at scale, we would want qualified mental-health professionals to review the language, low-risk action library, escalation flow, and evaluation plan."

If someone may be in immediate danger, direct them to local emergency services or a trusted person. For India, the Ministry of Health and Family Welfare lists Tele-MANAS at 14416 and 1-800-891-4416. Confirm current availability and local resources before presenting them as live support. [National Mental Health Programme / Tele-MANAS]

Do not say keyword detection is reliable, that it prevents suicide, or that the app monitors a user continuously.

8. Live demo walkthrough

Run the app from the repository configuration and verify both services are healthy before presenting. Use your actual configured frontend/API URLs; do not hard-code a port in a slide unless you checked it on that machine.

Recommended route sequence:

1. Home (/) - explain the user problem and the single-loop promise.
2. Sign in / sign up - show the account boundary only with a demo account; never put real credentials on screen.
3. Dashboard - complete Focus, Energy, and Stress check-in. Explain the score and drivers as estimates.
4. Recommendation - explain why one action was selected. Start or skip it.
5. Measurement - complete the follow-up and helpfulness feedback. Point out the observed change and explain the no-action estimate cautiously.
6. History / Insights (/history, /insights) - show that the loop was recorded and how prior user feedback informs later choices.
7. Privacy (/privacy) - describe the controls and actual storage split honestly.
8. Demo (/demo) - optional: use seeded personas and time controls to make a repeatable short presentation. Clearly say the personas are demo data.

The other available pages include /onboarding, /profile, /login, and /signup. Avoid relying on exact score, action, confidence interval, or net-effect numbers: these may vary with data and state.

90-second spoken demo

> "MaxxLoop starts with a quick check-in about focus, energy, and stress. It compares that check-in with the user's recent pattern and explains the main contributors. Instead of handing an overloaded user a long list, it offers one small action. The user can try it, check in again, and say whether it felt helpful. That response becomes part of the user's recommendation history. The score and measured change are product estimates, not clinical measurements or proof of causality. This demo persona is seeded data; we still need real-world evaluation to assess impact."

Then complete one actual loop on screen. If a stage fails, narrate the failure honestly and use the demo console only if the issue is clearly due to live data/setup; do not claim a successful live action that did not complete.

9. Likely judge questions and good answers

"Is this a medical app?"
"No. It is a non-medical wellness and focus-support prototype. It does not diagnose or treat conditions. Its capacity score is an estimate, and its safety phrase matching is limited."

"What is AI about it?"
"The recommendation layer personalizes from recorded feedback using a lightweight Thompson-sampling approach. The default text layer is deterministic templates. We do not need to pretend a generative model is making the core score or that one is active in this demo."

"How do you know the action worked?"
"We collect a self-reported before/after check-in and compare it with an estimated no-action drift based on prior windows. It helps us track observed response, but it is not randomized evidence or proof of causation."

"Where is data stored?"
"Authentication accounts and sessions use MongoDB. Activity and recommendation records use the API's SQLite database. In a hosted deployment, storage and retention would need production configuration and clear disclosure."

"What happens with crisis language?"
"A limited English/Hindi/Kannada phrase scanner can interrupt the normal recommendation and show support resources. It can miss language or context and is not an emergency monitoring service."

"What is your evidence?"
"We have a working software flow and synthetic simulations for exercising the recommendation logic. Those simulations are not real-user studies. The next evidence step is supervised usability work and an appropriately reviewed study."

"How will it scale?"
"The current prototype separates the web client, API, auth store, and product records. Before broader deployment we would address managed persistent storage, operational monitoring, security review, privacy disclosures, and a reviewed safety process. Those are production work, not claims about the current local demo."

10. Evidence and limitations

The repository contains a synthetic simulation of 200 generated users over 30 rounds (6,000 simulated loops). The accompanying evaluation document reports simulation metrics such as helpfulness and capacity change. These are simulation outputs, not participant results, independent verification, a clinical study, or evidence that MaxxLoop improves mental health. If you mention the numbers, put "synthetic simulation" beside them on the slide and in your spoken explanation.

Current limitations to acknowledge when relevant:

- No established clinician partnership, clinical validation, or real-user efficacy study.
- Crisis matching relies on a limited phrase list and can miss context, language, spelling, or indirect expressions.
- Before/after self-report and estimated drift do not demonstrate causation.
- The default text provider is template-based; optional external model providers require explicit configuration and data-flow review.
- MongoDB and SQLite need appropriate managed/backup/persistent configuration before a public production deployment. Do not say this project is production-hosted unless it has actually been deployed and verified.
- Direct calendar, wearable, or browser-history integrations should only be described as existing if you can demonstrate their working implementation. Do not imply that the app reads private sources automatically based on the roadmap or old slides.

11. Final preparation checklist

- Confirm the frontend and API start with the project’s current configuration and show healthy status.
- Use a demo account and seeded persona; do not display secrets or personal data.
- Rehearse one complete check-in -> recommendation -> action -> measurement -> feedback loop.
- Confirm the exact labels and buttons in the running UI before putting them on slides.
- Keep one backup screenshot or short recording in case the live environment fails; label it as a recording.
- Keep simulation numbers visibly labeled as synthetic.
- State the non-medical boundary and measurement limitations plainly.
- Keep the pitch focused: problem, working loop, why the feedback matters, how you protect the user, and what evidence you still need.

12. Implemented versus planned

Implemented in the current repository

- Email/password signup, login, logout, and current-user endpoint with MongoDB-backed accounts/sessions.
- Next.js pages for landing, auth, dashboard, demo, insights, history, profile, onboarding, and privacy.
- Self-report check-in, capacity/baseline estimate, driver labels, drop/cooldown checks, action selection, timer controls, follow-up measurement, feedback, and aggregate insights.
- SQLModel product records and privacy export/delete/consent endpoints.
- Deterministic template explanation provider; optional provider adapters exist in code.

- Authenticated synthetic demo seeding/drop/timewarp endpoints.

Not currently established as implemented or validated

- Public production deployment or production operations.
- Clinician partnership, clinical review, diagnostic validity, or health outcomes.
- Real-user efficacy evidence; simulation output is synthetic.
- Guaranteed crisis detection, emergency monitoring, or guaranteed safety.
- Automatic calendar, browser history, or wearable collection unless a working integration is separately demonstrated.
- Device-only or zero-knowledge storage, or a legal compliance certification.

Planned / future work

Usability evaluation, security/privacy review, expert safety review, improved measurement and evaluation, production-ready durable storage/operations, and optional integrations with explicit consent. These are goals, not current features; see [Roadmap].

13. Competition positioning and presentation discipline

Position MaxxLoop around a complete user journey and an inspectable prototype architecture. The defensible innovation claim is the project’s integration of check-in, personal-baseline estimate, one action, follow-up, and feedback-driven recommendation state. Avoid "first," "only," "proven," "clinically safe," or "guaranteed improvement" unless independently substantiated.

The ASYNC’26 official Wellness & Lifestyle prompt describes turning everyday signals into understanding/improvement and the Track -> Understand -> Act -> Measure -> Improve pattern. The official page lists idea-deck requirements and published idea-stage weights; those should not be represented as a confirmed final-round rubric unless organizers confirm it. [Problem statements / criteria] • [Rules].

For exact click order, use [Demo Script]; for challenging questions, use [Judge Q&A].

MaxxLoop Live Demo Script (3-5 minutes)

Use a test account and synthetic demo persona. Check the running buttons and API health before starting; exact scores/actions can vary. Never show real passwords, personal records, or environment values. A demo seed resets the shared user_default dataset, so use demo mode only.

Time	What to click	What to say	What the judge should notice
0:00-0:25	Open / and introduce MaxxLoop	"MaxxLoop connects a short self-check-in to one useful next step and a follow-up measurement. We want to close the gap between tracking and action."	Clear problem/solution; describe it as a prototype, not a medical tool.
0:25-0:50	Open /signup and create or use a prepared demo account; then /dashboard	"Authentication uses email/password. For the demo I use a test account; the auth store is MongoDB and product records are stored separately by the API."	Actual account flow and dashboard. Do not reveal credentials.
0:50-1:20	Complete Focus, Energy, Stress check-in; submit	"These are self-reported inputs. The API compares them with available history and returns an estimated capacity score and contributing signal labels."	Short form, range validation, loading/error response, explanation.

1:20-1:50	Show drivers and the action/reason panel if one is offered	"When the configured drop and cooldown rules allow it, the system offers one action matched to the active drivers. Selection uses stored per-action Beta priors and Thompson sampling; the default explanation is deterministic template text."	Single action and inspectable context. A drop is not guaranteed from every input.
1:50-2:20	Start the action; show timer/steps. If a demo drop is needed, use /demo controls.	"The user can start or skip. The action window is normally configured for 25 minutes; demo timewarp can shorten it for presentation."	Controls correspond to persisted intervention state. Label any seeded data as synthetic.
2:20-3:10	Complete follow-up Focus/Energy/Stress and submit measurement; send helpful/not helpful feedback	"We record observed self-report change and subtract an estimated no-action drift from prior skipped windows, with a default prior when data is sparse. This is a useful prototype measure, not proof of causation."	Actual before/after flow and honest caveat.
3:10-3:40	Open /history, then /insights	"These views summarize saved signals and outcomes. Helpful ratios and net-effect summaries describe this user's recorded data, not clinical efficacy."	Persisted records and summary views.
3:40-4:10	Open /privacy; show consent/export/delete controls without executing destructive action	"Activity records are server-side in SQL storage, while accounts and sessions use MongoDB. The template provider is the default; any external provider must be configured and its data flow disclosed."	Correct privacy/storage explanation. Do not call it device-only or zero-knowledge.
4:10-4:30	Conclude	"Our current contribution is the closed user loop. Next we need usability evaluation, stronger safety review, and evidence from real users before making impact claims."	Clear implementation-versus-roadmap distinction.

Recovery plan if live demo stalls

1. Confirm /health, MongoDB availability for auth, and frontend/API URLs.
2. If account creation fails, use a previously created test account; do not show an error as a success.
3. If a check-in does not trigger a drop, explain that recommendation is conditional. Use the authenticated demo console's synthetic trigger only if appropriate.
4. If demo controls fail, continue explaining the actual architecture and be explicit that the live interaction did not complete. A prerecorded clip/screenshot must be identified as prerecorded.

Judge-safe phrases

- Say "estimated capacity score," not "clinical stress score."
- Say "observed self-report difference minus estimated drift," not "true treatment effect."
- Say "synthetic demo history," not "customer data."
- Say "personalization heuristic," not "validated AI model."

Judge Q&A

Answers below reflect the current repository. They are intentionally candid. Add team-specific facts only if you can verify them.

Product

1. What is MaxxLoop?

Answer: A focus/recovery prototype that connects a short check-in to an

explanation, one suggested action, follow-up measurement, and feedback.
Technical detail: The loop is implemented across /checkins, /loop/*, and /insights.

2. Who is it for?

Answer: Initially students and knowledge workers who want lightweight support for focus and recovery.
Technical detail: Current account model is general; no institution/role-specific access control is implemented.

3. What problem are you solving?

Answer: Self-tracking can show a state without a next step, while generic advice may not fit the user's context. We connect a check-in to one action and a later record.

4. What is the main differentiator?

Answer: The end-to-end closed loop: check in, understand, try one action, measure again, and use feedback to shape later suggestions. We do not claim no competitor does this.

5. Why one action?

Answer: The product is designed to reduce choice burden in the moment. The API creates a single offered intervention for an eligible drop.

6. What does the user see after a check-in?

Answer: An estimated capacity score/status, baseline and change, available drivers, and an action if the drop and cooldown rules permit.

Technical

7. What is the stack?

Answer: Next.js/React/TypeScript/Tailwind on the web; FastAPI/Python/Pydantic/SQLModel on the API; SQLite for product data and MongoDB for auth.

8. How does the frontend reach the API?

Answer: A shared fetch wrapper uses NEXT_PUBLIC_API_URL (default localhost:8000), JSON, and credentialed requests for the session cookie.

9. How is the capacity score computed?

Answer: Signal values are compared with baselines, standardized and weighted, then mapped to a bounded 0-100 product score. It is not a clinical measurement.
Technical detail: See engine/baseline.py, capacity.py, weights.yaml, and [Technical architecture].

10. What happens at cold start?

Answer: The baseline engine uses configured population priors/fallback blending when personal history is sparse, then can use more personal history as it accumulates.

11. How are drivers selected?

Answer: Driver contributions from the capacity calculation are labeled and ranked; up to three are passed into the recommendation path.

12. What database tables exist?

Answer: User profile, signal, capacity snapshot, intervention, outcome, and action-prior tables are defined in SQLModel. Mongo has users and sessions collections.

13. How does user-data isolation work?

Answer: Auth dependencies resolve the account ID and product queries use that ID. Demo operations deliberately use a shared user_default dataset and should stay in demo-only use.

14. What protects passwords?

Answer: The auth helper hashes passwords with Argon2 and stores the hash, not plaintext. We have not represented that as a complete security audit.

15. How are sessions handled?

Answer: A random opaque token is set in an HTTP-only cookie; Mongo stores a SHA-256 hash with expiry. The session is checked on protected requests and removed on logout.

AI / ML

16. Is this machine learning?

Answer: It is not a trained ML model. The action selector uses a lightweight statistical Beta/Thompson-sampling method over stored outcomes.

17. Is generative AI required?

Answer: No. The default provider is deterministic templates. Optional external providers exist, but must be configured and their data flow disclosed.

18. How are actions selected?

Answer: Candidate actions are filtered by target-driver overlap. The engine samples Beta values and usually chooses the highest sample, with a 15% exploration branch when multiple candidates exist.

19. How does feedback change recommendations?

Answer: Measurement updates alpha when net effect exceeds two points, otherwise beta. The explicit helpfulness response is stored separately and is not the prior update criterion in the current code.

20. Does the algorithm prove which action works?

Answer: No. It adapts a heuristic from observed in-app data. Selection bias, self-report, small samples, and the drift estimate limit interpretation.

Data and measurement

21. How do you measure an action?

Answer: Compare pre/post self-report scores, then subtract an estimated no-action drift based on skipped-intervention windows, blended with a prior when sparse.

22. Is net effect causal?

Answer: No. It is an observational estimate and not a randomized controlled comparison.

23. What data do you collect automatically?

Answer: Do not claim automatic calendar, wearable, or browser collection unless you can demonstrate a working source integration. Current check-in inputs are

self-reported; signal endpoints can accept numeric signals.

24. Where is data stored?

Answer: Auth accounts/sessions are in MongoDB. Product activity and profile/preferences are stored server-side in SQLModel storage, SQLite by default.

25. Is demo data real?

Answer: No. Demo seeding creates synthetic persona history and sample records.

Safety, privacy, and security

26. Is MaxxLoop a medical product?

Answer: No. It does not diagnose or treat conditions, and its score is not clinically validated.

27. What happens when crisis language is entered?

Answer: A limited phrase scanner can pause the normal check-in flow and return support resources. It can miss meaning and is not emergency monitoring.

28. Is a doctor part of the team/product?

Answer: There is no clinician built into the current product and no clinical review is established by the repo. Professional review is a future requirement before any clinical positioning.

29. Is data zero-knowledge or device-only?

Answer: No such claim is supported. Product data is stored by the backend; accounts/sessions use MongoDB.

30. Can users export or delete data?

Answer: The API has export and product-data deletion endpoints. Deletion retains account/session infrastructure and profile while deleting activity records and priors; it is not account deletion.

Scalability, business, innovation, and limits

31. Is it deployed publicly?

Answer: No verified production deployment is documented in this repository. There is a Dockerfile/Compose setup, but Compose does not include MongoDB and needs configuration review.

32. How would you scale it?

Answer: First establish managed durable storage, deployment secrets/networking, observability, backups, rate limits, and a reviewed security/privacy process. Then measure real workload before scaling architecture.

33. What evidence do you have?

Answer: There is an implemented software flow and a synthetic simulation. Simulation output is not user evidence or clinical validation.

34. What is the business model?

Answer: The current code does not implement billing or validate a business model. A possible future path could be explored with users/institutions, but that is a hypothesis.

35. What is the biggest risk?

Answer: Users may interpret an estimated score or limited recommendation as more

authoritative than it is. Clear non-medical language, careful safety review, and real-user evaluation are essential.

36. What would you build next?

Answer: Usability and API/browser tests, a privacy/security review, clinician-informed safety review, and a careful real-user evaluation before claims of effectiveness.

Limitations

- Prototype evidence: no real-user efficacy study or clinical validation is established. Synthetic simulation is not participant data.
- Medical boundary: the score is not validated clinically; the product does not diagnose or treat conditions.
- Crisis safety: phrase matching is limited, can miss indirect language/context and can false-positive. It is not continuous monitoring or guaranteed protection.
- Measurement: self-report before/after plus estimated drift cannot prove causation; fallback drift is used when control history is sparse.
- Recommendation: priors and the threshold-based update are simple heuristics. Feedback volume, selection bias, and synthetic/demo records can affect apparent results.
- Data integrations: repository signal kinds support manual/import-style numeric inputs, but automatic calendar, wearable, or browser-history integration should not be claimed without a working implementation.
- Privacy: product data is stored server-side in SQL DB; auth is in Mongo. No privacy certification or legal compliance assessment is documented.
- Operations: Compose omits Mongo and no verified production deployment configuration was found. SQLite durability needs explicit volume/backup planning.
- Auth features: no password reset, email verification, or account deletion route was found.
- Testing: backend tests exist; no browser automation or frontend test runner configuration was found in the inspected project files.
- Demo: seeded persona uses shared user_default data, resets that demo dataset, and includes generated synthetic history. Keep it separate from real users.

Roadmap

Everything below "Current" is future work and not a claim about implemented capability.

Current

- Account/session flow with MongoDB.
- Check-in, capacity estimate, driver display, one-action recommendation, measurement, feedback, insights/history, privacy controls, and authenticated demo tools.
- SQLite product persistence by default; template explanation provider by default.

Next (suggested)

- Rehearse moderated usability sessions and improve confusing steps.
- Add frontend/browser tests and API contract coverage for critical flows.
- Review privacy disclosures, consent semantics, data retention, and account lifecycle.

- Conduct security review of production session, CSRF, rate limit, secret, and deployment settings.
- Improve measurement methodology before making efficacy claims.
- Add operational health/readiness checks and durable database backups.

Future (ideas, not commitments)

- Qualified clinical/safety advisor review of language, action library, and crisis handoff.
- Carefully designed real-user evaluation with appropriate review and consent.
- Optional integrations (calendar/wearables) only with explicit consent and minimization.
- Mobile client or improved installable experience.
- More robust experimentation and recommendation evaluation.
- Managed SQL/Mongo deployment, observability, scaling, and operational playbooks.